

Long-Form Football Video Automation Guide

Complete Agent-Executable Workflow

This document instructs an AI agent on how to produce a fully automated, fact-checked, narrated long-form football YouTube video — from brief to public upload — replicating exactly the process used to build *"Okay, This England Squad Is SCARIER Than You Think..."* on 2026-06-18.

Machine	Windows 11 Pro 10.0.22621, 8GB RAM
Python	3.12 (python / py)
Shell	PowerShell (primary) + Git Bash available
FFmpeg	C:\Users\Wendy\AppData\Roaming\Python\Python312\site-packages\imageio_ffmpeg\binaries\ffmpeg-win-x86_64.exe
yt-dlp	Installed in PATH (run: yt-dlp --version to confirm)
Working user	Wendy

Table of Contents

1. Project Folder Structure
2. Secrets & Credentials
3. Python Dependencies
4. Phase 1 — Research & Fact-Checking
5. Phase 2 — Script Writing
6. Phase 3 — B-Roll & Insert Sourcing (yt-dlp)
7. Phase 4 — Frame Probing Inserts
8. Phase 5 — Build Script Setup & Configuration
9. Phase 6 — Running the Build
10. Phase 7 — QC Frame Inspection
11. Phase 8 — YouTube Upload
12. Phase 9 — Slack Notification
13. Known Gotchas & Hard-Won Rules
14. Quick-Reference Checklists

1. Project Folder Structure

Each video gets its own subfolder under the Long Form Automation project root. Create the folder before starting. The England video folder was:

```
C:\Users\Wendy\Claude\Projects\Long Form Automation\england_tuchel_scarier\
```

The folder structure inside each video project is always:

```
england_tuchel_scarier/  
  assets/  
    broll/          <- all downloaded b-roll and insert MP4 files  
    vo/             <- generated voiceover MP3s (one per section)  
    music_bed.mp3   <- background music (copied from sibling project)  
  _tmp/             <- FFmpeg segment work files + QC frames  
  final_video/      <- assembled final MP4  
  script_*.md       <- the written + fact-checked VO script  
  build_*.py        <- the main build script  
  yt_upload_*.py    <- the YouTube upload script
```

NOTE: The build script auto-creates all subdirectories at runtime. You only need to create the top-level project folder.

The sibling project root (for copying music_bed.mp3 if missing) is:

```
C:\Users\Wendy\Claude\Projects\Long Form Automation\ronaldo_portugal_problem\assets\music_  
bed.mp3
```

2. Secrets & Credentials

WARNING: NEVER hardcode API keys in scripts or command lines. Always read them from the .env.local file described below.

2.1 Environment Variables (.env.local)

All API keys live in:

```
C:\Users\Wendy\Documents\Football Channel\work\secrets\.env.local
```

The file format is key=value, one per line. Required keys:

ELEVENLABS_API_KEY	ElevenLabs API key for TTS generation
ELEVENLABS_VOICE_ID	Voice ID to use (default: bu5eKETbFKC8G702EAU4 — the established channel voice)
SLACK_BOT_TOKEN	Slack bot token for #new-videos notifications (if used directly)

The build script loads these automatically with this pattern (always copy this into new build scripts):

```
SECRETS = Path(r'C:\Users\Wendy\Documents\Football Channel\work\secrets\.env.local')

def load_env():
    if SECRETS.exists():
        for line in SECRETS.read_text().splitlines():
            line = line.strip()
            if line and not line.startswith('#') and '=' in line:
                k, v = line.split('=', 1)
                os.environ.setdefault(k.strip(), v.strip())

load_env()
ELEVEN_API_KEY = os.environ.get('ELEVENLABS_API_KEY', '')
VOICE_ID       = os.environ.get('ELEVENLABS_VOICE_ID', 'bu5eKETbFKC8G702EAU4')
```

2.2 YouTube OAuth Token

The YouTube upload token (OAuth2 refresh token) is at:

```
C:\Users\Wendy\Documents\Football Channel\work\secrets\yt_publish_token.json
```

This token was obtained by running the OAuth flow once manually with google-auth-oauthlib. It refreshes automatically using the refresh_token field. If it expires or fails, the user must re-run the OAuth flow — this cannot be automated headlessly.

To use it in an upload script:

```
from google.oauth2.credentials import Credentials
from google.auth.transport.requests import Request

creds = Credentials.from_authorized_user_file(str(TOKEN))
if creds.expired and creds.refresh_token:
    creds.refresh(Request())
```

3. Python Dependencies

The build script self-installs missing packages at runtime via pip. However, for reliability, pre-install all of these before the first run:

```
pip install requests pillow google-api-python-client google-auth-oauthlib google-auth-http  
lib2
```

requests	ElevenLabs TTS API calls
Pillow (PIL)	Thumbnail PNG→JPEG compression (YouTube 2MB limit)
google-api-python-client	YouTube Data API v3 upload + thumbnail
google-auth-oauthlib	OAuth2 credential refresh
google-auth-http2	HTTP transport for Google APIs
imageio-ffmpeg	Bundles FFmpeg binary — already installed on this machine

FFmpeg is invoked via its full path to avoid PATH issues on Windows. The path is:

```
C:\Users\Wendy\AppData\Roaming\Python\Python312\site-packages\imageio_ffmpeg\binaries\ffmp  
eg-win-x86_64-v7.1.exe
```

Always detect the FFmpeg path with a helper function:

```
FFMPEG_FALLBACK = r'C:\Users\Wendy\AppData\Roaming\Python\Python312\...\ffmpeg-win-x86_64-  
v7.1.exe'  
  
def ffmpeg_exe():  
    return FFMPEG_FALLBACK if Path(FFMPEG_FALLBACK).exists() else 'ffmpeg'
```

4. Phase 1 — Research & Fact-Checking

WARNING: This is the most critical phase. Two factual errors have already shipped to YouTube on previous videos. Every specific claim — match numbers, records, statistics, scorelines — **MUST** be verified with a live web search before writing the VO script. Never rely on training data for live match results.

4.1 What to Research

For a World Cup / tournament video, always verify:

- Exact final scoreline and goalscorers (with minutes)
- Whether this is the team's FIRST, SECOND, etc. match in the tournament — very easy to get wrong
- Any records broken or equalled — verify the current record holder and exact figure
- Starting XI and substitutes who scored
- Player availability / injury context (who was rested, managing fitness)
- Qualifying / historical stats being cited (wins, goals conceded, etc.)
- Tournament context (group, venue, date)

4.2 Confirmed Facts for the England Video (as a worked example)

Match	England 4-2 Croatia, Group L, June 17 2026, AT&T Stadium, Dallas/Arlington TX
Scorers	Kane x2 (pen + header from Rice corner), Bellingham (47'), Rashford (sub, exact minute unconfirmed)
Croatia scorers	Baturina + Musa — 2-2 at HT; England dominant in 2nd half
Starting XI	Pickford; James, Stones, Konsa, O'Reilly; Rice, Bellingham; Madueke, Anderson, Gordon; Kane
Saka status	Benched — still managing Achilles recovery, described as 'not 100%'
Kane WC goals	10 — equalling Gary Lineker's all-time England WC record (needs 1 more to break it)
Bellingham	Youngest European at 4 major tournaments (22y 353d)
Kane 2025-26	58 goals in all competitions for Bayern; 3rd consecutive Bundesliga Golden Boot
Left out	Cole Palmer, Phil Foden, Trent Alexander-Arnold, Harry Maguire
Qualifying record	8W 0D 0L, 0 goals conceded — 720 min Pickford clean sheet; first European nation to qualify

5. Phase 2 — Script Writing

5.1 Video Structure

The 9-section structure used for long-form videos (5-6 minutes) is:

Section 1 — Hook (VO)	~28s — immediate hook on the match result, set up the thesis
Section 2 — Insert A	~22s — pundit reaction / pre-match footage from b-roll insert clip
Section 3 — Identity (VO)	~36s — what has changed under this manager vs predecessor
Section 4 — Ruthless (VO)	~39s — the ruthless decision / defining moment that sets the tone
Section 5 — Depth (VO)	~47s — squad depth, all the weapons available
Section 6 — Insert B	~22s — manager press conference / squad announcement footage
Section 7 — Weapons (VO)	~37s — individual star quality — records, stats, recent form
Section 8 — Proof (VO)	~39s — historical / qualifying proof this isn't a fluke
Section 9 — Outro (VO)	~44s — final summary, stakes, call to action

5.2 VO Writing Rules

- Target ~120-130 words per minute of narration at 1.2x speed — so aim for ~110 words per 60s of section
- Write in second person / direct address: 'here is what you missed', 'let's talk about'
- Never start with 'I' or 'We' — start with the subject or a question
- Each section should end on a beat that bridges to the next (Insert A follows Hook, so hook should tease 'but what did the experts say?')
- Keep sentences short — this is spoken audio, not prose
- Do not reference 'this video' or 'subscribe' in the VO — no meta-commentary
- Total VO word count target: 800-900 words across 7 sections for a ~5-6 minute video

5.3 Script File

Save the completed script as a markdown file in the project root:

```
C:\Users\Wendy\Claude\Projects\Long Form Automation\script_.md
```

Mark it FINAL once all facts are verified. This file is for reference only — the build script embeds the VO text directly in the SECTIONS list.

6. Phase 3 — B-Roll & Insert Sourcing (yt-dlp)

6.1 Types of Clips Needed

Main match b-roll	Fan compilation of the actual match — goals, highlights. Target 640x360 or 1280x720.
Secondary b-roll	Training footage, kit reveal, player features — same team. Used to vary the visuals.
Insert A	Pundit reaction / pre-match content from a credible football channel (Football Daily, Sky Sports e
Insert B	Manager press conference — ideally where they explain controversial selection decisions

6.2 Source Vetting Rules

WARNING: Inspect b-roll sources carefully BEFORE committing to them. Common problems: baked-in black letterbox borders, AV1 codec that crashes FFmpeg, baked-in chyrons or subscribe buttons, cutaway to unrelated content mid-clip.

After downloading, probe each clip:

```
# Check duration and codec
ffprobe -v quiet -print_format json -show_format -show_streams clip.mp4

# Extract sample frames
ffmpeg -ss 30 -i clip.mp4 -vframes 1 -update 1 probe_30s.jpg
ffmpeg -ss 180 -i clip.mp4 -vframes 1 -update 1 probe_180s.jpg
```

Visually inspect the frames. Reject the clip if:

- Black bars are baked into the frame (letterbox) — crop-fill cannot remove these
- Source watermarks cover more than 15% of the frame
- The codec is AV1 (libdav1d) — FFmpeg on this machine may crash
- The clip is from official FIFA/UEFA/Premier League — high Content ID risk

6.3 Downloading with yt-dlp

WARNING: Chrome v127+ uses App-Bound DPAPI encryption that breaks yt-dlp cookie extraction. Firefox is not installed on this machine. The ONLY working method is the android player client bypass shown below.

```
yt-dlp --extractor-args "youtube:player_client=android" \
-f 18 \
-o "assets/broll/filename.mp4" \
"https://www.youtube.com/watch?v=VIDEO_ID"

# Format 18 = 360p MP4 (H264+AAC) – fast to process, plenty for 1080p upscale
# If format 18 not available, use: -f 'best[height<=480][ext=mp4]'
```

After download, always record the confirmed duration:

```
ffprobe -v quiet -show_entries format=duration -of csv=p=0 clip.mp4
```

You need the exact duration (in seconds) to set safe b-roll seek offsets in the build script. An offset that exceeds the clip duration will produce a silent/black segment.

6.4 Confirmed Clips for England Video

broll_england_croatia_2026.mp4	638s, 50.3MB, 640x360 H264 — fan highlights, ENG vs CRO
broll_england_kit_reaction.mp4	815s, 59.9MB, 640x360 H264 — Kane/Bellingham/Saka England kit reveal

insert_football_daily_england_day.mp4	877s, 57.3MB, 640x360 H264 — Football Daily BBC pundit duo, live at stadium
insert_tuchel_squad_announcement.mp4	3080s, 134.8MB, 640x360 H264 — full Tuchel squad announcement press conference

7. Phase 4 — Frame Probing Inserts

Insert clips are long source videos — you must find the exact timecode window containing the pundit/manager speaking clearly before setting the start/end times in the build script.

7.1 Extract Probe Frames

```
# For Insert A – probe at 30s, 90s, 180s, 300s, 480s
ffmpeg -ss 30 -i insert_football_daily_england_day.mp4 -vframes 1 -update 1 _tmp/probe_insertA_30s.jpg
ffmpeg -ss 180 -i insert_football_daily_england_day.mp4 -vframes 1 -update 1 _tmp/probe_insertA_180s.jpg
ffmpeg -ss 480 -i insert_football_daily_england_day.mp4 -vframes 1 -update 1 _tmp/probe_insertA_480s.jpg

# For Insert B – probe at 300s, 600s, 900s, 1200s, 1800s, 2400s
ffmpeg -ss 300 -i insert_tuchel_squad_announcement.mp4 -vframes 1 -update 1 _tmp/probe_insertB_300s.jpg
ffmpeg -ss 900 -i insert_tuchel_squad_announcement.mp4 -vframes 1 -update 1 _tmp/probe_insertB_900s.jpg
```

7.2 What to Look For

Insert A (pundit)	Pundit(s) clearly visible, speaking to camera, no transition/cutaway happening in the frame
Insert B (manager)	Manager in close-up or mid-shot, clearly at a press conference with relevant backdrop
Both inserts	Full frame fill (no black bars), content topically relevant to the section it follows

7.3 Timecodes Found for England Video

Insert A start/end	00:00:30 to 00:00:52 (22s) — Football Daily duo on-mic at Dallas stadium
Insert B start/end	00:15:00 to 00:15:22 (22s) — Tuchel close-up speaking directly to camera

NOTE: Target 20-25 seconds per insert for optimal retention. The build script applies a 0.8s audio+video fade-out at the end of each insert automatically.

8. Phase 5 — Build Script Setup & Configuration

8.1 Constants to Set at the Top of Every Build Script

```
VO_SPEED          = 1.2          # atempo - 1.2x faster narration for retention
VO_VOLUME_DB      = 10.0         # dB boost applied to VO after speed
INSERT_FADEOUT_SECS = 0.8        # audio+video fade-out at end of insert clips
BROLL_CLIP_MAX_SECS = 5.5        # max duration of each b-roll micro-clip
W, H              = 1920, 1080    # output dimensions

FILL_VF = f'scale={W}:{H}:force_original_aspect_ratio=increase,crop={W}:{H},fps=30'
# This crop-fill upscales 640x360 source to fill 1920x1080 without letterboxing
```

8.2 SECTIONS List Structure

The SECTIONS list is the heart of the build script. It defines all 9 sections in order, with types 'vo' or 'insert':

```
SECTIONS = [
    # VO section:
    {'type': 'vo', 'name': 'hook',
     'text': 'The voiceover text goes here...',
     'broll': [
         (MATCH, 10, 'hook1'), # (source_file, seek_offset_secs, unique_label)
         (MATCH, 60, 'hook2'),
         (KIT, 20, 'hook3'),   # alternate between MATCH and KIT for variety
         (MATCH, 120, 'hook4'),
         (KIT, 80, 'hook5'),
         (MATCH, 180, 'hook6'),
     ]},
    # Insert section:
    {'type': 'insert', 'name': 'insertA', 'src': INSERT_A_SRC,
     'start': '00:00:30', 'end': '00:00:52'},
]
```

8.3 Critical Rule: B-Roll Seek Offsets Must Not Exceed Clip Duration

WARNING: If a seek offset (e.g. 660) exceeds the clip's actual duration (e.g. 638s), FFmpeg will produce a silent/black segment and the build will silently fail. Always cap seek offsets at $(\text{clip_duration} - \text{BROLL_CLIP_MAX_SECS} - 1) = \text{clip_duration} - 6.5$.

MATCH (england_croatia_2026.mp4)	638s — max safe offset: 631
KIT (england_kit_reaction.mp4)	815s — max safe offset: 808
Insert A (football_daily_england_day.mp4)	877s — only 22s extracted from 00:00:30
Insert B (tuchel_squad_announcement.mp4)	880s — only 22s extracted from 00:15:00

8.4 Audio Rules (Never Break These)

- ALWAYS use `-ac 2 -ar 44100` on every FFmpeg audio output
- NEVER use `-shortest` to trim a voiceover — always extend video to match audio length first (the `overlay_audio` function handles this)
- NEVER burn captions into the video — user edits captions in CapCut and exports separately
- Music bed is mixed at -35dB (background only, barely audible under narration)

9. Phase 6 — Running the Build

9.1 Pre-Run Checklist

- ✓ All 4 clip files exist in assets/broll/ and have been probed for duration
- ✓ All b-roll seek offsets are within safe range for their source clip
- ✓ Insert A and Insert B timecodes have been verified by frame inspection
- ✓ TRAINING variable does not exist — use a descriptive name like KIT or TRAINING_2024
- ✓ ELEVENLABS_API_KEY is present in .env.local and not empty
- ✓ music_bed.mp3 is available (auto-copied from sibling project if missing)

9.2 Running the Script

```
cd 'C:\Users\Wendy\Claude\Projects\Long Form Automation\'  
python build_.py
```

The script will:

Probe both b-roll sources and print their durations

Auto-copy music_bed.mp3 from a sibling project if not present

For each VO section: generate ElevenLabs TTS, speed up 1.2x, build b-roll track, overlay audio

For each insert section: trim the source clip to the specified window + fade

Concatenate all 9 segments

Mix in music bed at -35dB

Run quality check: verify 1920x1080, <=600s, stereo audio

Extract 9 QC frames (one per section) to _tmp/qc_*.jpg

Print a JSON summary including section timecodes

9.3 ElevenLabs VO Generation

Settings used for this channel's voice:

```
{  
  'model_id': 'eleven_multilingual_v2',  
  'voice_settings': {  
    'stability': 0.42,  
    'similarity_boost': 0.82,  
    'style': 0.18,  
    'use_speaker_boost': True  
  }  
}  
  
# Voice ID: bu5eKETbFKC8G702EAU4 (set in ELEVENLABS_VOICE_ID in .env.local)
```

NOTE: ElevenLabs can hallucinate filler sounds at the START of the audio (e.g. a breath or 'um'). The build script writes a 2-second spot-check MP3 for each VO section (vo_*_check2s.mp3 in assets/vo/). If you hear filler at the start of any section, delete that VO file and re-run the build — it will regenerate only the missing ones.

10. Phase 7 — QC Frame Inspection

After the build completes, you MUST visually inspect all 9 QC frames before uploading. QC frames are saved to:

```
C:\Users\Wendy\Claude\Projects\Long Form Automation\\_tmp\qc_*.jpg

# One frame per section:
qc_hook.jpg, qc_insertA.jpg, qc_identity.jpg, qc_ruthless.jpg,
qc_depth.jpg, qc_insertB.jpg, qc_weapons.jpg, qc_proof.jpg, qc_outro.jpg
```

10.1 What to Check in Each Frame

No letterboxing	Black bars at top/bottom or left/right mean the crop-fill failed — do NOT upload
Full 1920x1080 fill	Video must fill the entire frame — no grey/black borders anywhere
Insert frames show talent	Insert A should show the pundit; Insert B should show the manager — not a wide room shot
Correct section content	Each frame should show content topically relevant to that section's VO
No frozen frame	If a frame is solid black or a single colour, the b-roll segment is broken

10.2 QC Pass Criteria for England Video

All 9 frames passed:

hook	Croatian player in warm-up — match b-roll, full frame
insertA	Football Daily pundits live at Dallas stadium — BBC/FIFA 2026 branding visible
identity	Bellingham + Kane in England kit reveal — full frame
ruthless	Match action ENG 3-0 CRO scoreboard visible — full frame
depth	England player in training gear at St George's Park — full frame
insertB	Tuchel close-up at press conference, England sponsor backdrop — full frame
weapons	England squad lined up for anthem — Kane and players visible
proof	Wide match shot ENG 2-0 CRO scoreboard — full frame
outro	England player in home kit — clean close-up, full frame

NOTE: The source match clip has a 'Subscribe' button and 'Football Prime' watermark baked in — these appear briefly during fast b-roll cuts and are acceptable. They cannot be removed without re-sourcing the footage.

11. Phase 8 — YouTube Upload

11.1 Thumbnail Preparation

WARNING: YouTube rejects thumbnails larger than 2MB (2,097,152 bytes). PNG thumbnails are almost always over this limit. Always compress to JPEG first.

The thumbnail for this video was at:

```
C:\Users\Wendy\Documents\Football Channel\output\imagegen\absurd-england-kane-saka-bellingham-19x9.png
# Size: 2.04MB — exceeded 2MB limit
# Compressed to JPEG at quality=90 -> 346KB -> saved to _tmp/thumbnail_*.jpg
```

Compression code (always include this in upload scripts):

```
from PIL import Image

def compress_thumbnail(src_png, dst_jpg):
    img = Image.open(src_png).convert('RGB')
    for quality in [90, 80, 70, 60]:
        img.save(dst_jpg, 'JPEG', quality=quality)
        if dst_jpg.stat().st_size < 2_097_152:
            return dst_jpg
    raise RuntimeError('Cannot compress below 2MB')
```

11.2 Upload Script Structure

Create a dedicated upload script per video named: yt_upload_.py

Key upload code:

```
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

request = youtube.videos().insert(
    part='snippet,status',
    body={
        'snippet': {'title': TITLE, 'description': DESCRIPTION,
                    'tags': TAGS, 'categoryId': '17'}, # 17 = Sports
        'status': {'privacyStatus': 'public',
                  'selfDeclaredMadeForKids': False},
    },
    media_body=MediaFileUpload(str(VIDEO), chunksize=5*1024*1024, resumable=True),
)

# Always use resumable upload for files >50MB
response = None
while response is None:
    status, response = request.next_chunk()

video_id = response['id']
```

11.3 Set Thumbnail After Upload

```
youtube.thumbnails().set(
    videoId=video_id,
    media_body=MediaFileUpload(str(thumb_jpg), mimetype='image/jpeg'),
).execute()
```

11.4 Running the Upload

```
cd 'C:\Users\Wendy\Claude\Projects\Long Form Automation\'  
python yt_upload.py
```

Expected output on success:

```
Uploaded! Video ID: 3SxvUMAS47k  
URL: https://www.youtube.com/watch?v=3SxvUMAS47k  
Setting thumbnail: thumbnail_england_tuchel_scarier.jpg (346 KB)  
Thumbnail set.  
=== DONE ===  
Status : public
```

NOTE: The QC-pass video MUST be set to public immediately on upload. Do not upload as private and forget to flip it — use `privacyStatus='public'` in the upload body.

12. Phase 9 — Slack Notification

After a successful upload, send a notification to the #new-videos Slack channel. The Slack MCP tool is available in the Claude Code session:

```
# Channel: #new-videos
# Channel ID: C0B7GER7E77
# Workspace: jeremy-cole-workspace.slack.com

# Send via slack_send_message MCP tool:
channel_id = 'C0B7GER7E77'
message = (
    'New video live!\n\n'
    f'***{TITLE}**\n'
    f'https://www.youtube.com/watch?v={video_id}\n\n'
    f'{duration_min} long  Public  Thumbnail set\n\n'
    'One-line description of the video content.'
)
```

13. Known Gotchas & Hard-Won Rules

- ***yt-dlp YouTube bot check (HTTP 429)***

Chrome v127+ DPAPI breaks cookie extraction. Firefox not installed. ONLY working fix: --extractor-args "youtube:player_client=android". Always use this flag.

- ***-shortest flag cutting VO audio***

NEVER use -shortest as the primary mux strategy. The overlay_audio() function extends video to audio_duration + 0.5s first, then muxes. -shortest is only used AFTER the extension, so it trims excess video, not audio.

- ***B-roll offsets exceeding clip duration***

If offset + BROLL_CLIP_MAX_SECS > clip_duration, FFmpeg produces a silent/black segment. The build does not error — it silently produces garbage. Always verify every offset is < (clip_duration - 6.5) before running.

- ***ElevenLabs filler at VO start***

ElevenLabs sometimes adds a breath or 'um' at the very start of generated audio. The build writes a 2s spot-check file (vo_*_check2s.mp3). Listen to each one. If filler is present, delete the full MP3 and re-run — it will regenerate.

- ***Thumbnail PNG over 2MB***

YouTube hard-rejects thumbnails > 2,097,152 bytes. PNG game thumbnails are typically 2-5MB. Always compress to JPEG at quality=90 first. If still over limit, try quality=80 then 70.

- ***OOM on captions.py (Whisper model)***

This machine has 8GB RAM. The default Whisper model (small.en) causes OOM. Use --model tiny.en. base.en can also fail. DO NOT use captions in this pipeline anyway — user adds captions in CapCut manually.

- ***No captions burned into video***

NEVER burn captions into the video output. User imports the raw video into CapCut, adds captions there, and exports to youtube_ready/. The upload script always uploads from final_video/, not youtube_ready/.

- ***Match number errors in script***

Two factual errors shipped to YouTube: Portugal's 'second' WC match (was first), and Messi 'one goal away from equalling' (had already equalled). Always web-search match numbers and records. Never write from memory.

- ***Video format rotation***

Do not make every video the 'Warned Us' format. Rotate through: Downfall, Was Right, Historical-figure, Reassessment. This video was a 'depth/analysis' format.

- ***Official match footage Content ID***

Never use official FIFA, UEFA, or Premier League match footage. Use fan compilations, training clips, kit reveals, press conferences. The england_croatia_2026.mp4 clip is a fan compilation ('Football Prime' watermark).

14. Quick-Reference Checklists

14.1 Before Writing the Script

- Web-searched: final scoreline and goalscorers confirmed
- Web-searched: match number in tournament (1st, 2nd, etc.)
- Web-searched: any records being cited — exact holder and figure
- Web-searched: player availability / injury context
- Identified 4 source clips (match, secondary, insert A, insert B)
- Probed all clips for duration and frame quality

14.2 Before Running the Build Script

- ELEVENLABS_API_KEY present in .env.local
- All 4 clip files in assets/broll/ — durations confirmed
- No b-roll seek offset exceeds (clip_duration - 6.5)
- Insert A timecodes verified by frame inspection (pundit visible)
- Insert B timecodes verified by frame inspection (manager visible)
- VO_SPEED = 1.2 set
- FILL_VF uses scale+crop for full-frame fill
- Audio: -ac 2 -ar 44100 on every FFmpeg output
- No -shortest used to trim VO

14.3 After Build — Before Upload

- Build completed with exit code 0
- specs_ok: true in JSON output
- Duration between 280s (4:40) and 600s (10:00)
- All 9 QC frames inspected — no letterboxing, no black frames
- Insert A frame shows pundit on-screen
- Insert B frame shows manager on-screen
- VO spot-check MP3s listened to — no filler at start

14.4 Upload Checklist

- Thumbnail file checked — if PNG, compress to JPEG (target <1MB)
- privacyStatus = 'public' in upload body
- Title matches the agreed video title exactly
- Upload completed with video_id returned
- Thumbnail set successfully
- Slack notification sent to #new-videos (channel ID: C0B7GER7E77)
- YouTube URL confirmed live and public

